Ticket Ninja 🥷

Software Requirements Specification

1

2/12/2026

Group 2

Jake Webb, Mark Guggenheim, Hector Estrada

Prepared for
CS 250- Introduction to Software Systems
Instructor: Dr. Gus Hanna
Fall 2024

Revision History

Date	Description	Author	Comments
2/19/26	Version 1	Group	First Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	<Your Name>	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces 3
3.1.2 Hardware Interfaces 3
3.1.3 Software Interfaces 3
3.1.4 Communications Interfaces 3
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> 3
3.2.2 <Functional Requirement or Feature #2> 3
3.3 Use Cases 3
3.3.1 Use Case #1 3
3.3.2 Use Case #2 3
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> 3
3.4.2 <Class / Object #2> 3
3.5 Non-Functional Requirements 4
3.5.1 Performance 4
3.5.2 Reliability 4
3.5.3 Availability 4
3.5.4 Security 4
3.5.5 Maintainability 4
3.5.6 Portability 4

4. Software Design Specification 5

4.1 UML Diagram

4.1.1 UML Diagram Description

4.2 SWA Diagram

4.2.1 SWA Description

1. Introduction

This SRS document details the requirements for the Ticket Ninja ticket manager software system.

1.1 Purpose

The purpose of this SRS document is to explain in detail the vision and purpose behind Ticket Ninja. It is intended for software engineers to read and understand what is necessary to build Ticket Ninja accurately based on the design specifications.

1.2 Scope

Ticket Ninja will be used as an interface for moviegoers to browse for and purchase tickets for movies on-location or online across all theaters in the U.S. It should have the capabilities to allow users to search for specific movies at specific locations, dates, and times, pick their seat(s), choose between discounted child, senior, or student tickets, or buy the standard price adult ticket(s), preorder food and drinks for the movie, and finally acquire payment information for a 3rd party payment processor.

In order to accomplish these goals, Ticket Ninja will be able to run on several web browsers, handle at least 5,000,000 requests simultaneously, use location information to find nearby theaters, communicate with the software used to handle food/drink orders at the desired theater location, generate a functionally infinite amount of unique ticket codes, and properly charge customers for their tickets and food/drink orders via cash (on-location machines only, USD) or digital payment accepted by our linked payment processor.

1.3 Definitions, Acronyms, and Abbreviations

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.
SRS-Software Requirements Specification
UI-User interface
API- Application programming interface
DB-Database
MTBF-Mean Time Between Failures

1.4 References

We haven't used any references at the moment. All information on this document has originated from our general knowledge and vision for the system.

1.5 Overview

This document outlines the requirements for the TicketNinja system.

Section 1 introduces the purpose and scope of the system and explains how this document is organized.

Section 2 provides a general description of TicketNinja, including what the system is intended to do, who will use it, and the overall constraints and assumptions that affect development.

Section 3 contains the detailed functional and non-functional requirements that will guide the design and implementation of the system.

Section 4 includes supporting analysis models such as diagrams that help visualize system behavior.

Section 5 describes how changes to this document will be handled throughout the project.

2. General Description

This section provides an overview of the TicketNinja system and the environment in which it will operate. It explains the overall purpose of the product, the types of users who will interact with it, and the general limitations or constraints that may affect development.

This section does not list specific technical requirements. Instead, it helps give context so that the detailed requirements in the next section are easier to understand.

2.1 Product Perspective

- Theater seat management systems
- Theater food ordering systems
- Third-party payment processors.
- Location services APIs

2.2 Product Functions

-User account creation and login
-Movie search and filtering
-Seat collection and reservation
-Ticketing pricing calculation

-Food and beverage pre-ordering
-Ticket generation and validation
-Email confirmation

2.3 User Characteristics

- Primary Users:

-General public(ages 13+)
-Basic Internet literacy
-Uses mobile or desktop browsers

- Secondary Users:

-Theater staff validating tickets
-System administrators

- Assumptions:

-Theater staff validating tickets
-Users can input payment information correctly.

2.4 General Constraints

System operates with basic web browsers (Firefox, Google Chrome, Safari, etc)
Follows proper third-party payment processor policies
System must support high traffic load during new movie releases
Must integrate smoothly with theater seating and food management system

2.5 Assumptions and Dependencies

-Third party payment API remains operational
-Theater APIs provide real-time seat availability
-Users have stable internet connection
-Hosting infrastructure supports required traffic load

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

-Provide responsive layout for desktop and mobile
-Display seat maps graphically
-Prevent selection section of already-booked seats
-Display seat maps graphically
-Provide confirmation page with ticket code
-Provide accessible color contrast and keyboard navigation

3.1.2 Hardware Interfaces

-Support standard web browsers
-Support touchscreen kiosks
-Support QR code scanners for tickets validation
-Support receipt printer(kiosk only)

3.1.3 Software Interfaces

-Payment processing API (Stripe or equivalent)
-Theater seat management API
-Theater food ordering API
-Email notification service API

3.1.4 Communications Interfaces

-All communication shall use HTTPs (TLS 1.2 or higher)
-Payment communications shall comply with PCI-DSS
-API communication must use JSON format
-System shall support geolocation services

3.2 Functional Requirements

3.2.1 <Functional Requirement or Feature #1>

3.2.1.1 Introduction

-Allows users to search and filter available movies.
3.2.1.2 Inputs
-Movie title (optional)
-ZIP code or location
-Date
-Time range

3.2.1.3 Processing
-Query database for matching showtime
-Filter result

3.2.1.4 Outputs
-List of available movies and showtimes
-Theater location details

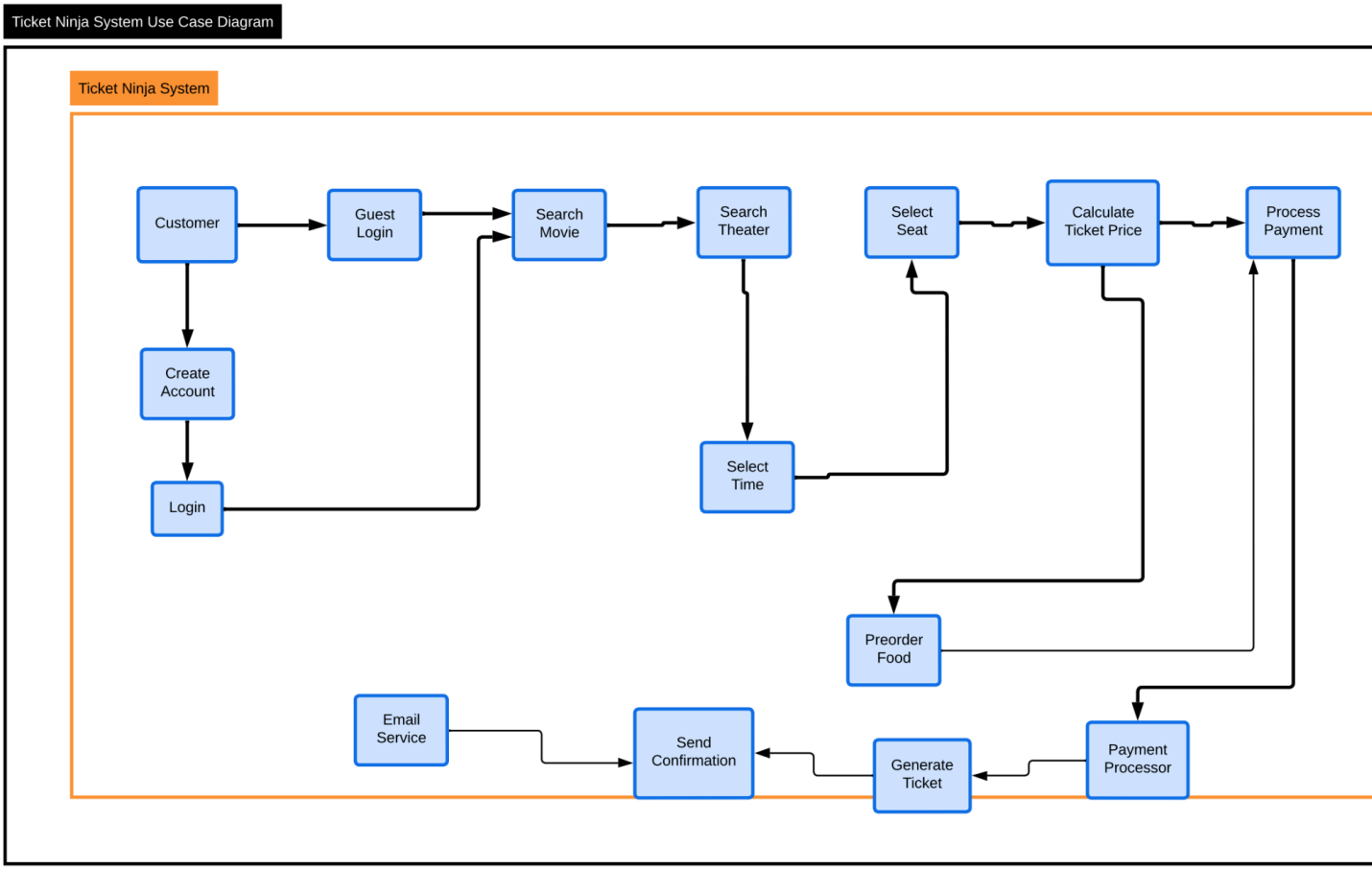
3.2.1.5 Error Handling
-Display message if no results found
-Validate location input format

3.2.2 <Functional Requirement or Feature #2>

…

3.3 Use Cases

3.3.1 Use Case #1: Online Ticket Ordering

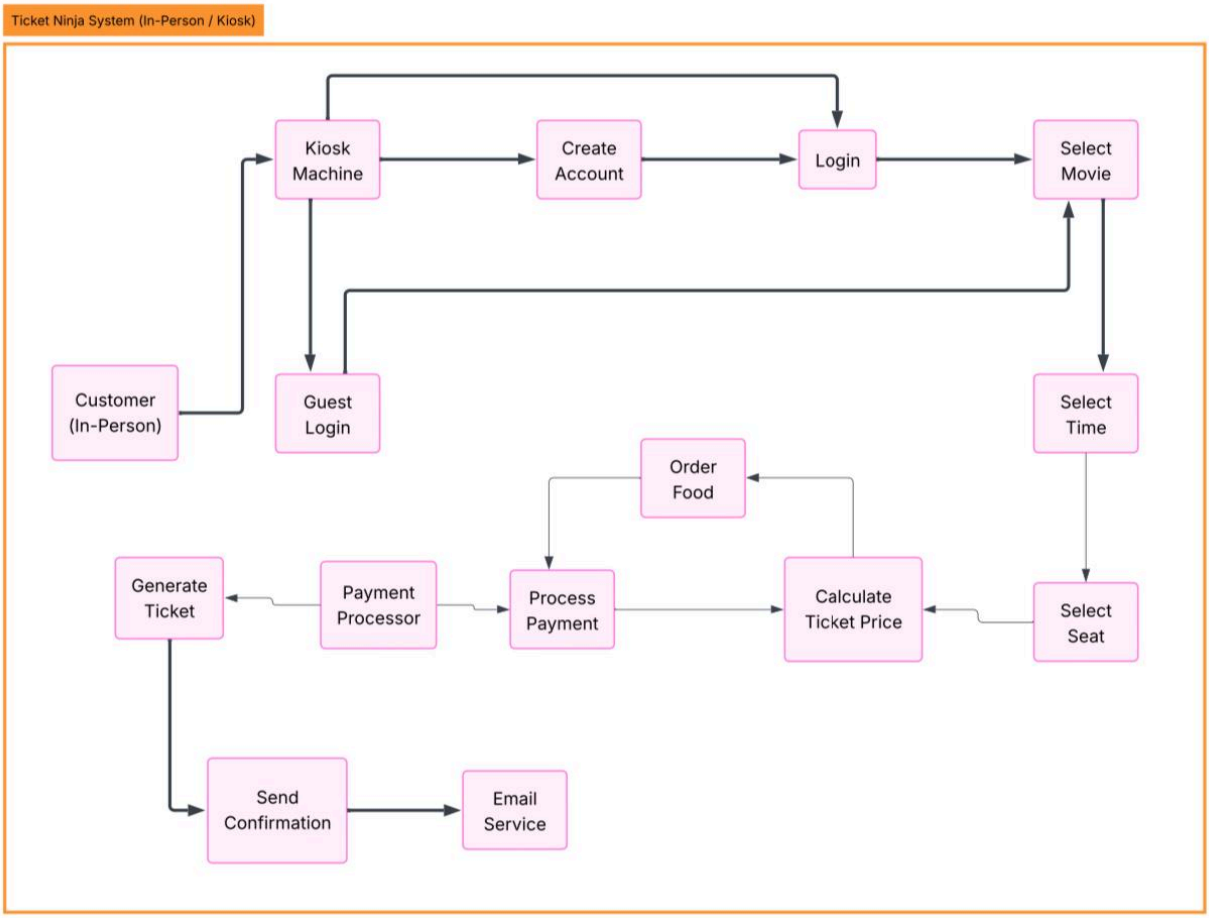


Actor: Customer
Steps:

1. User logs in/uses guest account
2. User selects location

3. User selects movie
4. User selects seats
5. User selects ticket type
6. User entering payment info
7. System validates payment
8. System generates ticket

3.3.2 Use Case #2: Ordering Tickets at a Theater



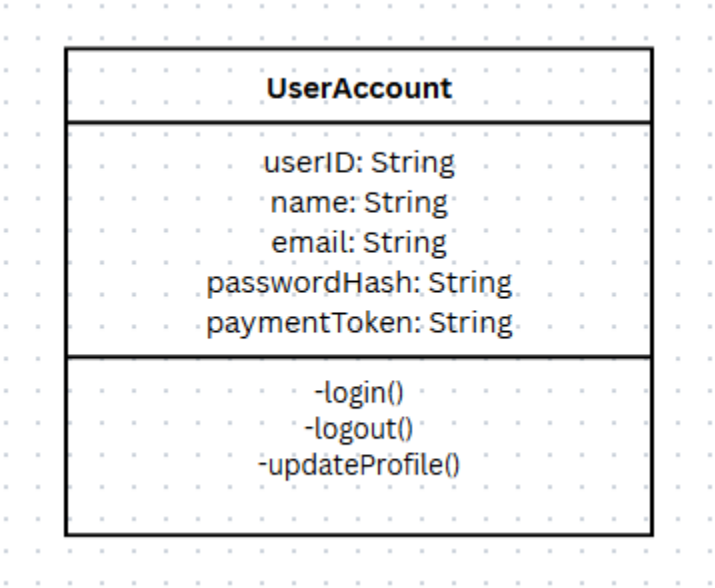
Actor: Customer
Steps:

1. User logs in/uses guest account on ticketing machine
2. User selects location
3. User selects movie
4. User selects seas
5. User selects ticket type
6. User entering payment info
7. System validates payment
8. System generates ticket and sends user an email with code/QR code

3.4 Classes / Objects

3.4.1 UserAccount

3.4.1.1 Attributes

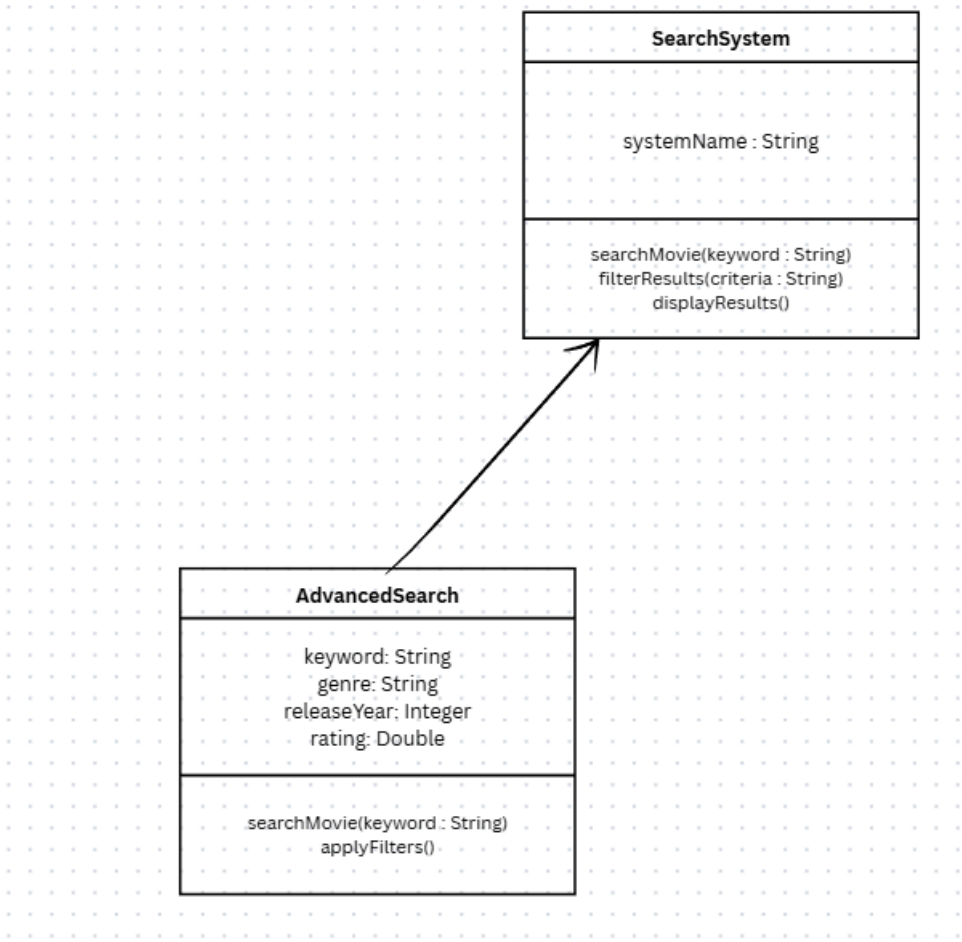


-userID
-name
-email
-passwordHash
-paymentToken

3.4.1.2 Functions
-login()
-logout()
-updateProfile()

<Reference to functional requirements and/or use cases>

3.4.2 Search System



3.4.2.1 Attributes
systemName
genre
releaseYear
rating

3.4.2.2 Functions
searchMovie(keyword)
filterResults(criteria)
applyFilters()
displayResults()

3.5 Non-Functional Requirements

3.5.1 Performance

- Must complete location handling and payment processing in less than 3 seconds

3.5.2 Reliability

- Must be able to handle 5 million concurrent requests/users

3.5.3 Availability

- Servers active 24/7, only down for maintenance, the times of which are communicated to users

3.5.4 Security

- Customer payment data should not be stored in case of breach
- System should not be prone to hacking
    - No backdoors or admin access given to theaters, all will be managed from Ticket Ninja
- Anti-scalping protection via turing test methods before reserving seats (ex. CAPTCHA)

3.5.5 Maintainability

- Service charge on all transactions large enough to sustain server expenses assuming X amount of customers per month

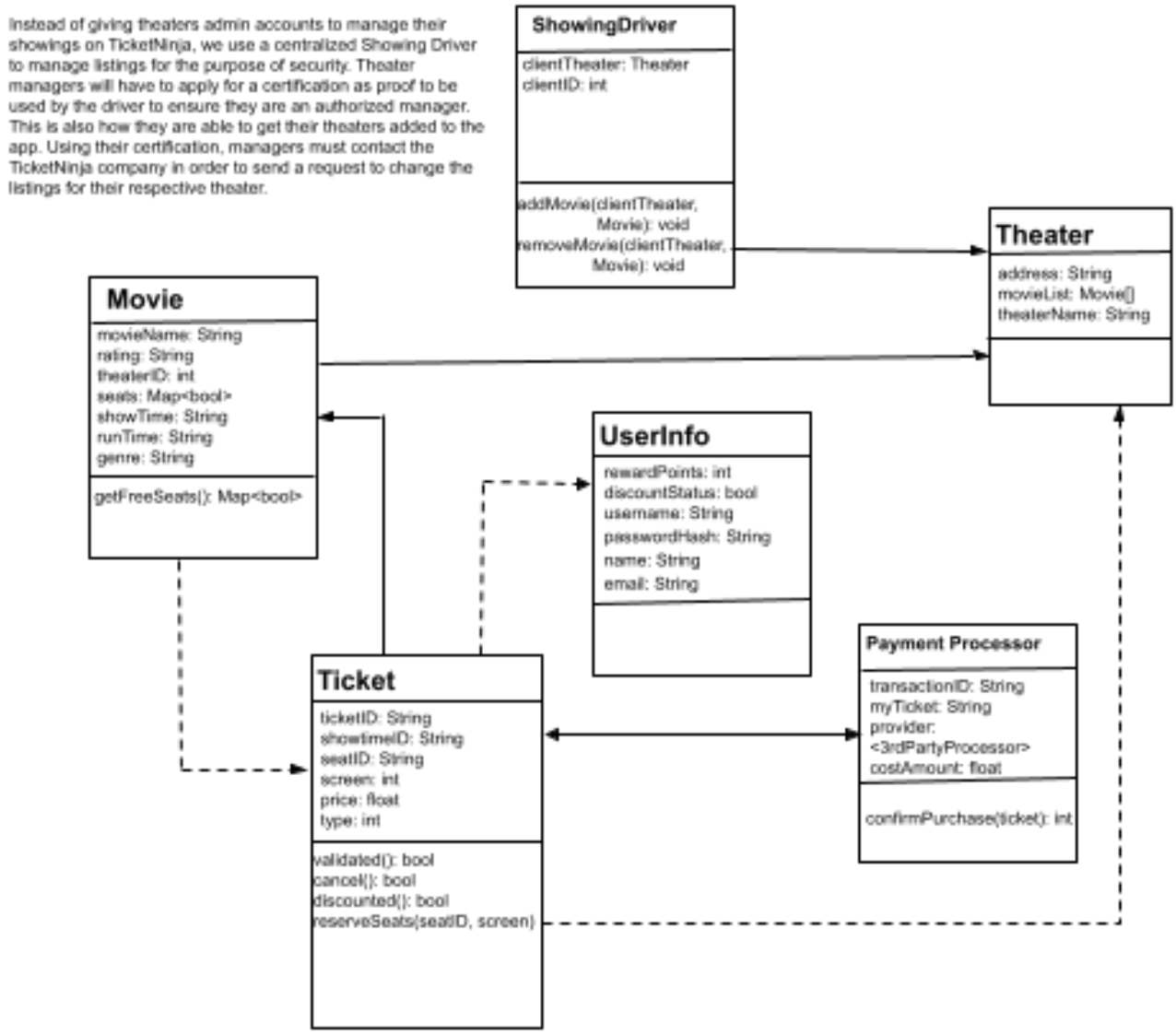
3.5.6 Portability

- Must be run as a web application, so that it may be used on all devices that can run an internet browser.

4. Software Design Specification

4.1 UML Diagram

Figure 1: UML Diagram

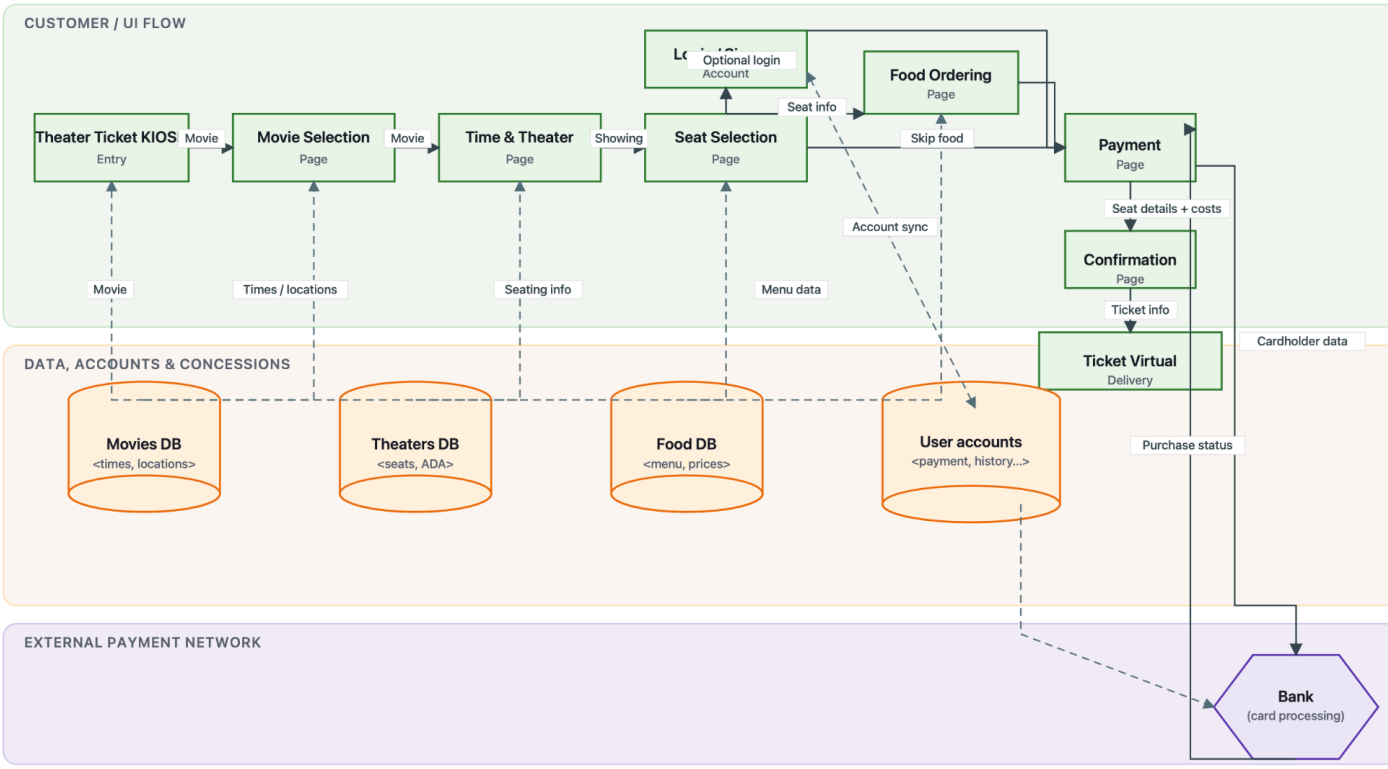


4.1.1 UML Description

This UML Diagram details how our classes will work together in order to form the TicketNinja system as a whole. We opted to not deal with location data in the sense we all know, and decided to instead allow theater managers to input all relevant information (city, address, theater name, etc.) when signing up for our service. The signing up and communication between managers and the TicketNinja system do not occur within the web application, and is managed elsewhere in the company. The main link from managers to TicketNinja is the Showing Driver, which takes in a request to edit theater showings from another system not included in the web app. The Theater

class stores all of the relevant location data and contains a list of movies playing. When users first connect to our web app, they will search for their desired theater before anything else. The Movie class contains lots of information about the movie, and keeps track of which seats in a particular screening are available. UserInfo stores account information and rewards points that are collected from using our service, and can be used to purchase tickets and food. The Ticket class stores all the information that will be sent out in an email after the purchase, and keeps track of discount status if the customer applies a discount. The Ticket class sends only the transaction data to the payment processor, which then connects with a third party payment processor system in order to properly charge the customer. This is also a security measure, since data breaches on our end will not reveal any payment information from our customers.

4.2 SWA Diagram



4.2.1 SWA Description

This SWA diagram shows the flows of information between various systems contained within TicketNinja. While it doesn't specify classes and fields like the UML diagram, the SWA more clearly shows the routes data takes between modules. For the most part, it is pretty linear starting from left to right. We have a few databases and third party programs that interact with modules in various places like the movie selection page and food ordering page.

4.3 Test Plan

GitHub Link

Overall, our tests are designed to specifically try edge cases for each of our modules, and all of them as a whole. In our test case sample, we try cases with the payment processor properly calculating purchase totals, applying discounts, reserving seats, and delivering tickets. All three search modules are tested as well; several tests involve searching by movie and theater + date. Finally, we test all of the modules to see if they can work properly in tandem, detailed descriptions of the test cases are as follows:

OrderTotal_1: This test checks the **Confirmation_Page_Module** to make sure the system calculates the correct total for one adult ticket when no discounts or concession items are added. It is being tested to confirm that basic ticket pricing works correctly before more complex pricing cases are considered.

OrderTotal_2: This test checks the **Confirmation_Page_Module** and the **Concessions portion of the order flow** to make sure the system correctly adds the price of two student tickets along with one popcorn and one drink. It is being tested to confirm that the system can handle multiple ticket types and extra items in the same order without miscalculating the total.

DiscountValidation_1: This test checks the **Payment_Page_Module** to make sure the system does not allow more than one discount type, such as student and senior, to be applied to the same ticket. It is being tested to confirm that ticket rules are enforced correctly and that invalid discount combinations are stopped before checkout.

SeatReservation_1: This test checks the **Seat_Selection_Page_Module** to make sure a seat that is already unavailable, held, or sold cannot be selected again by another customer. It is being tested to prevent double-booking and to confirm that seat availability is updated and enforced correctly.

ShowtimeSearch_1: This test checks the **Time_And_Theater_Page_Module** to make sure the system returns the correct theaters and showtimes when a user searches by location, movie title, and date. It is being tested to confirm that users can find valid movie options based on the filters provided.

ShowtimeSearch_2: This test checks the **Time_And_Theater_Page_Module** to make sure the system handles a valid search with no available results by showing a clear no-results message. It is being tested to confirm that the search feature responds properly when no matching showtimes exist and does not confuse the user with incorrect results.

CheckoutFlow_1: This test checks the **Confirmation_Page_Module**, including the **payment processor**, **ticket generation**, and **confirmation flow**, to make sure a customer can complete checkout successfully when payment is approved. It is being tested to confirm that the main purchase process works correctly across the connected parts of the system.

CheckoutFlow_2: This test checks the **Confirmation_Page_Module** and the **payment processor connection** to make sure the system responds correctly when a payment is declined. It is being tested to confirm that checkout is stopped, the purchase is not completed, and no ticket is generated after a failed transaction.

WebPurchaseFlow_1: This test checks the **Web Client system** (all modules) and its connection to the search, seat selection, checkout, and confirmation features to make sure a customer can complete the full ticket purchase process online. It is being tested because the browser-based workflow is one of the main ways users interact with Ticket Ninja.

KioskPurchaseFlow_1: This test checks the **Kiosk system** (also all modules) and its connection to ticket selection, checkout, and ticket output to make sure a customer can complete the full purchase process at a theater kiosk. It is being tested to confirm that the kiosk workflow works from start to finish and produces a valid ticket for theater entry.

4.3.1 Test Case Samples

(original document linked to table)

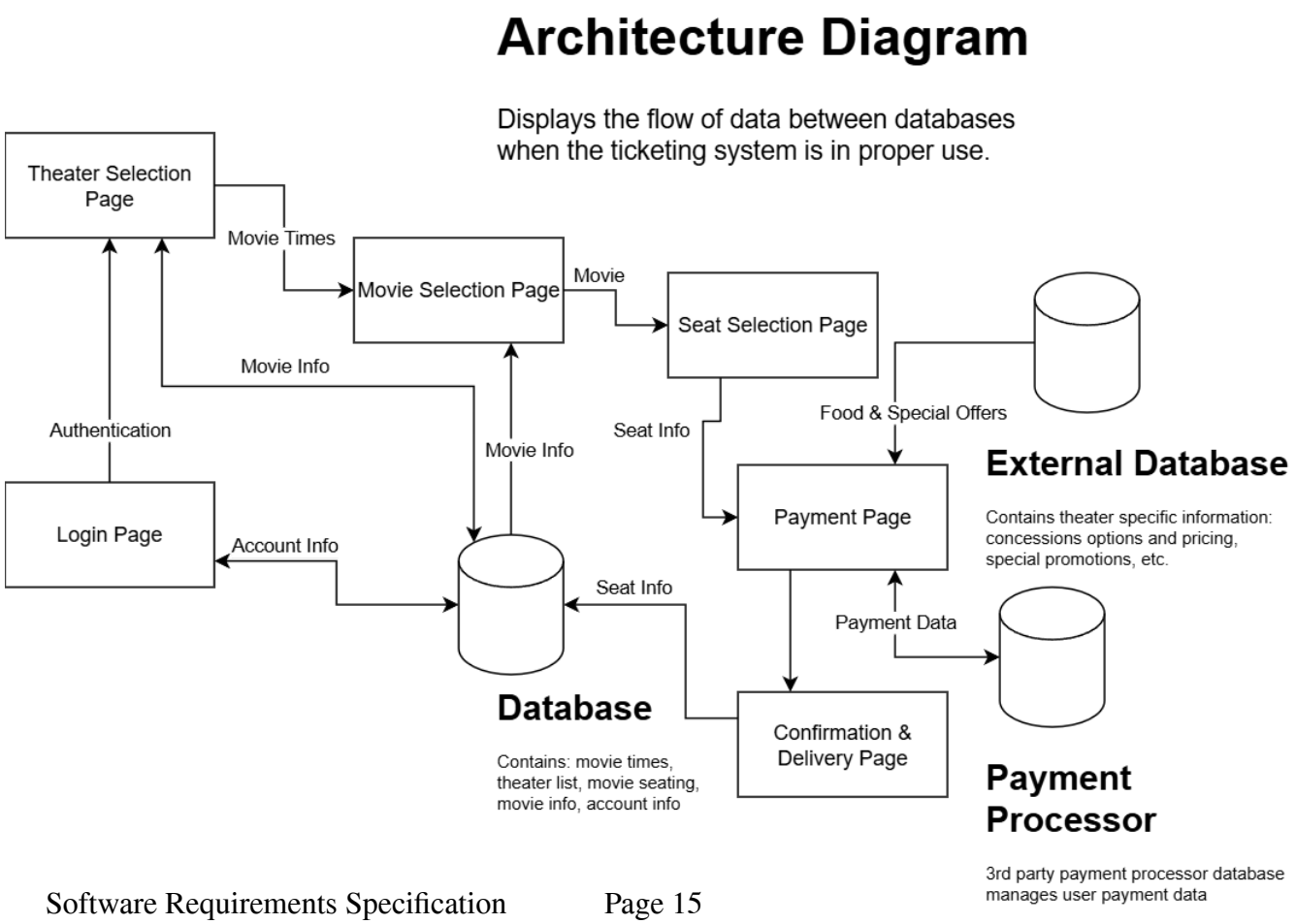
Test Case Template									
TestCaseId	**Component**	**Priority**	**Description/Test Summary**	**Pre-requisites**	**Test Steps**	**Expected Result**	**Actual Result**	**Status**	**Test Executed By**
OrderTotal_1	Confirmation_Page_Module	P0	Verify correct total for one adult ticket with no concessions or discounts.	A valid theater, movie, and showing are available in the system, and at least one seat is marked available for selection.	1. Use the search system to select a theater location, movie, and showing. 2. Select one available seat. 3. Create one Adult ticket for the selected seat. 4. Do not apply discounts or add food items. 5. Proceed to the pricing step and verify the total.	Order total equals adult ticket base price plus one service fee; seat remains held until checkout completes.	TBA	**TBA**	TBA
OrderTotal_2	Confirmation_Page_Module	P0	Verify mixed-ticket pricing and concession subtotal display and calculation.	A valid theater, movie, and showing are available in the system, at least two seats are marked available, and concession items are enabled for ordering.	1. Use the search system to select a theater, movie, and showing. 2. Select two available seats. 3. Create two Student tickets for the selected seats. 4. Add one popcorn and one drink to the order. 5. Proceed to the pricing step and verify the total.	Subtotal includes two student tickets, two service fees, and both concession items with correct arithmetic.	TBA	**TBA**	TBA

DiscountValidation_1	Payment_Page_Module	P1	Verify that incompatible discounts cannot be stacked on one ticket.	A valid movie showing and seat are available, and the system includes discount validation rules for Student and Senior ticket types.	1. Create a ticket for a valid movie showing and seat. 2. Assign Student status to the ticket. 3. Assign Senior status to the same ticket. 4. Attempt to continue to checkout. 5. Verify that the system rejects the invalid discount combination.	System rejects the invalid discount combination and prevents checkout until corrected.	TBA	**TBA**	TBA
SeatReservation_1	Seat_Selection_Page_Module	P0	Verify double-booking protection for a held or sold seat.	A valid movie showing exists with at least one seat already marked as unavailable, held, or sold.	1. Open a movie showing where one seat is already unavailable. 2. Attempt to select the same unavailable seat for a new order. 3. Submit the seat reservation request. 4. Verify that the reservation is rejected. 5. Verify that the seat remains unavailable.	System rejects the reservation request and preserves the original seat state.	TBA	**TBA**	TBA
ShowtimeSearch_1	Time_And_Theater_Page_Module	P0	Verify that search by theater, date, and title returns matching showtimes.	The system contains theater, movie, and showing data that matches a valid location, title, and date search.	1. Open the search system. 2. Enter a valid theater location, movie title, and date. 3. Submit the search request. 4. Verify that matching theaters and showings are displayed.	Relevant theaters and showtimes are displayed for the selected search criteria.	TBA	**TBA**	TBA
ShowtimeSearch_2	Time_And_Theater_Page_Module	P1	Verify no-results behavior for a valid search that has no matching showtimes.	The search system is available, and the entered location and date are valid but have no matching showings for the selected movie title.	1. Open the search system. 2. Enter a valid location and date with a movie title that has no matching showings. 3. Submit the search request. 4. Verify that the system displays a no-results message.	System displays a clear no-results message and does not crash or show unrelated results.	TBA	**TBA**	TBA
CheckoutFlow_1	Confirmation_Page_Module	P0	Verify successful checkout with seat hold, payment approval, ticket creation, and confirmation.	A valid theater, movie, and showing are available, one or more seats are open for selection, and the payment processor is configured to approve the transaction.	1. Log in or continue as a guest, then select a theater, movie, and showing. 2. Select available seats and assign ticket types. 3. Optionally add food or beverage items. 4. Enter valid payment information and submit checkout. 5. Verify that payment succeeds and tickets are generated..	Seats transition to Sold, payment is captured, ticket is generated, and a confirmation message is sent.	TBA	**TBA**	TBA
CheckoutFlow_2	Confirmation_Page_Module	P0	Verify rollback when the payment processor declines a transaction.	Sandbox payment gateway returns decline; seat is initially available.	1. Log in or continue as a guest, then select a theater, movie, showing, and seat. 2. Create the ticket and proceed to checkout. 3. Enter payment information that will be rejected. 4. Submit the transaction. 5. Verify that payment fails and no ticket is generated.	No ticket is issued, order remains unpaid or failed, and the held seat is released back to Available.	TBA	**TBA**	TBA

WebPurchaseFlow_1	All of them	P0	Verify the full browser-based purchase flow from search to confirmation.	The web application is accessible, valid movie and theater data are loaded, seats are available for purchase, and online checkout is enabled.	1. Open the Ticket Ninja web application. 2. Log in or continue as a guest, then search for a theater, movie, and showing. 3. Select seats and ticket types. 4. Enter payment information and complete checkout. 5. Verify that ticket confirmation is generated for the web user.	User completes an end-to-end online purchase and receives a valid ticket code or QR code.	TBA	**TBA**	TBA
KioskPurchaseFlow_1	All of them	P0	Verify kiosk purchase flow and ticket validation at theater entry.	The kiosk interface is operational, valid movie and theater data are loaded, seats are available for purchase, and ticket validation is enabled at theater entry.	1. Open the kiosk interface. 2. Select a location, movie, showing, seats, and ticket types. 3. Enter payment information and complete checkout. 4. Verify that the system generates a ticket code or QR code. 5. Verify that the ticket can be validated at theater entry.	Kiosk flow completes successfully and the generated ticket validates at entry.	TBA	**TBA**	TBA

4.4 Architecture Design and Data Management

4.4.1 Architecture Design Diagram



4.4.2 Data Management Strategy

Ticket Ninja uses a SQL-based data management strategy because the system handles structured information such as users, movies, theaters, seats, orders, and tickets. A relational database is a good fit because these parts of the system are connected and need to stay accurate, especially during ticket purchases and seat reservations.

Our design splits data into four main areas: movie and showtime data, theater and seat data, food and concession data, and user/order/ticket data. We chose to separate the data this way so the system is easier to organize, update, and maintain. For example, movie schedules can be updated without affecting user account information, and concession items can be managed separately from ticket purchases.

The movie and showtime data stores information about movies, dates, and times. The theater and seat data stores theater locations, seat layouts, and seat availability. The food and concession data stores menu items and prices for each theater. The user/order/ticket data stores account details, order history, rewards points, and generated ticket information.

Ticket Ninja does not store full payment card information. Instead, payment is handled by a third-party payment processor, and the system only stores basic transaction information such as payment status and confirmation details. This improves security and reduces risk if the system is ever compromised.

We considered putting all data into one database, which would be simpler at first, but separating the data into logical sections makes the system easier to manage as it grows. We also considered a non-SQL approach, but SQL fits this project better because the data is structured and the relationships between users, seats, showtimes, and tickets are important.

The decision to not manage location data and separate the payment processing from the TicketNinja system is for passive security. By not collecting any location data, our servers getting hacked poses no risk of sensitive user data being compromised. Keeping the payment processing as a third party system furthers this end by similar means. So by never storing any sensitive data in the first place, we avoid all risk of sensitive data leaks (and most lawsuits). The alternative is to build these systems ourselves and have better security shielding the whole system, but this can prove costly and inefficient when scaling up, so we opted for a simpler option that requires less overhead.